

Guía de Métodos para Listas en Python

Referencia completa y práctica con ejemplos paso a paso

Nota de Referencia: Las listas en Python (`list`) son estructuras de datos mutables y ordenadas. Todos los métodos presentados a continuación se invocan directamente sobre la instancia de la lista mediante la sintaxis `lista.metodo()`.

1. Métodos para Agregar Elementos

`append(x)`

INSERCIÓN

Agrega un único elemento `x` al final de la lista. Modifica la lista original y no devuelve ningún valor (retorna `None`).

```
# Crear una lista inicial
frutas = ["manzana", "banana"]

# Agregar un elemento al final
frutas.append("pera")

print(frutas) # Salida: ['manzana', 'banana', 'pera']
```

`extend(iterable)`

INSERCIÓN

Extiende la lista agregando todos los elementos provenientes de un objeto iterable (otra lista, tupla, conjunto, etc.) al final.

```
numeros = [1, 2]
mas_numeros = [3, 4, 5]

# Extender la lista con los elementos de otra
numeros.extend(mas_numeros)

print(numeros) # Salida: [1, 2, 3, 4, 5]
```

`insert(i, x)`

INSERCIÓN

Inserta el elemento `x` en una posición/índice específico `i`. Los elementos posteriores se desplazan hacia la derecha.

```
letras = ["a", "c", "d"]

# Insertar "b" en el índice 1
letras.insert(1, "b")

print(letras) # Salida: ['a', 'b', 'c', 'd']
```

2. Métodos para Eliminar Elementos

remove(x)

ELIMINACIÓN

Busca y elimina la **primera aparición** del valor x en la lista. Lanza un error ValueError si el elemento no existe.

```
colores = ["rojo", "azul", "verde", "azul"]

# Elimina la primera instancia de "azul"
colores.remove("azul")

print(colores) # Salida: ['rojo', 'verde', 'azul']
```

pop([i])

ELIMINACIÓN

Elimina y **devuelve** el elemento en el índice i. Si no se especifica el índice, elimina y devuelve el último elemento.

```
equipos = ["PC", "Laptop", "Tablet"]

# Eliminar y extraer el elemento en el índice 1
eliminado = equipos.pop(1)

print(eliminado) # Salida: 'Laptop'
print(equipos) # Salida: ['PC', 'Tablet']
```

clear()

ELIMINACIÓN

Elimina **todos** los elementos de la lista, dejándola completamente vacía.

```
datos = [10, 20, 30, 40]

# Vaciar la lista
datos.clear()

print(datos) # Salida: []
```

3. Métodos de Búsqueda y Conteo

index(x, [start], [end])

BÚSQUEDA

Devuelve el índice basado en cero de la primera aparición del elemento x. Lanza ValueError si no se encuentra.

```
nombres = ["Ana", "Luis", "María", "Luis"]

# Buscar la posición de "María"
pos = nombres.index("María")

print(pos) # Salida: 2
```

count(x)

CONTEO

Devuelve el número total de veces que el elemento x aparece dentro de la lista.

```
notas = [10, 7, 8, 10, 9, 10]

# Contar cuántos '10' hay en la lista
total_dieces = notas.count(10)

print(total_dieces) # Salida: 3
```

4. Ordenamiento y Utilidades

sort(key=None, reverse=False)

ORDEN

Ordena los elementos de la lista en su lugar (*in-place*). Permite personalizar el criterio mediante key y la dirección con reverse.

```
valores = [42, 12, 89, 7]

# Ordenar de menor a mayor
valores.sort()
print(valores) # Salida: [7, 12, 42, 89]

# Ordenar de mayor a menor
valores.sort(reverse=True)
print(valores) # Salida: [89, 42, 12, 7]
```

reverse()

ORDEN

Invierte el orden de los elementos de la lista directamente (*in-place*), sin crear una nueva lista.

```
serie = [1, 2, 3, 4]

# Invertir la lista
serie.reverse()

print(serie) # Salida: [4, 3, 2, 1]
```

copy()

COPIA

Devuelve una copia superficial (*shallow copy*) de la lista. Modificar la copia no afecta la lista original.

```
original = ["A", "B", "C"]

# Crear una copia independiente
copia = original.copy()
copia.append("D")

print(original) # Salida: ['A', 'B', 'C']
print(copia)    # Salida: ['A', 'B', 'C', 'D']
```